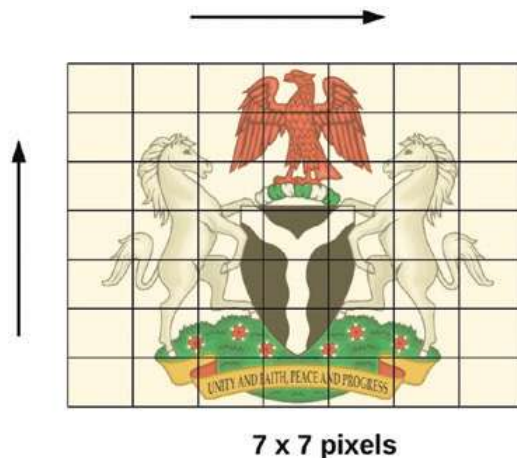


## CHAPTER 35

# Convolutional Neural Networks (CNN)

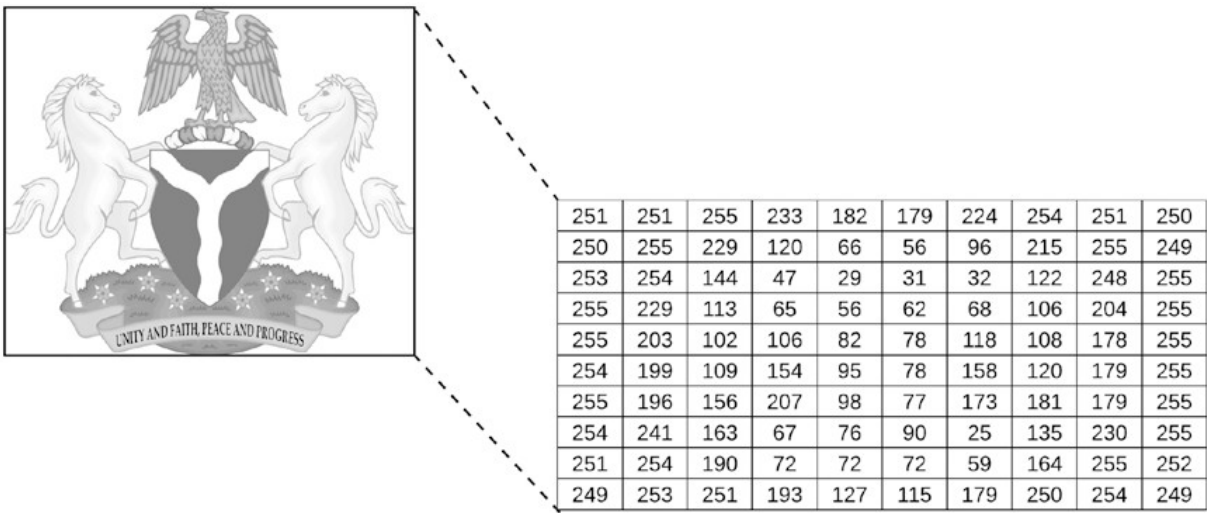
Convolutional neural networks (CNN) are a specific type of neural network systems that are particularly suited for computer vision problems such as image recognition. In such tasks, the dataset is represented as a 2-D grid of pixels. See Figure 35-1.



**Figure 35-1.** 2-D representation of an image

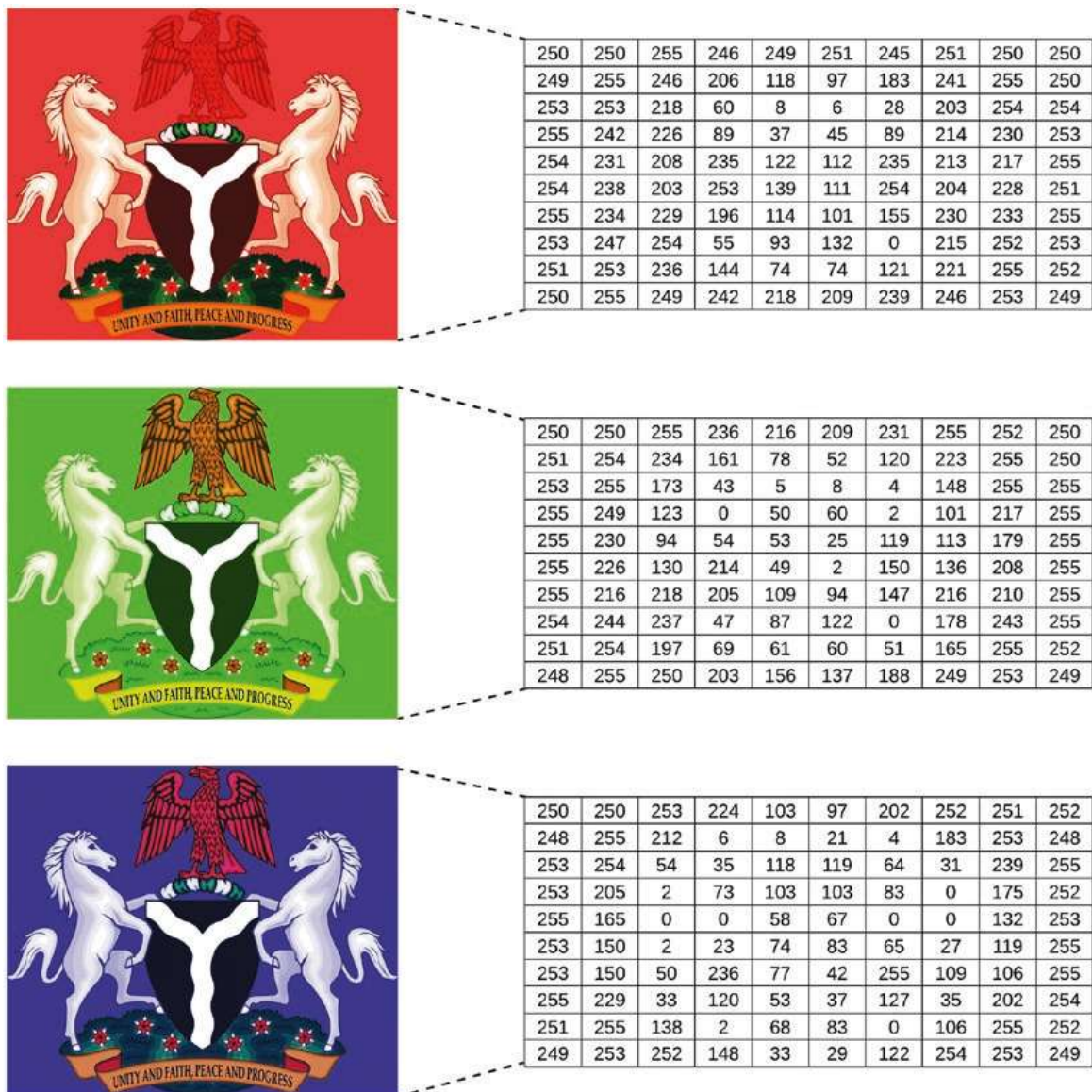
An image is depicted in the computer as a matrix of pixel intensity values ranging from 0 to 255. A grayscale (or black and white) image consists of a single channel with 0 representing the black areas and 255 the white regions with the values in between for various shades of gray.

For example, the image in Figure 35-2 is a 10 x 10 grayscale image with its matrix representation.



**Figure 35-2.** Grayscale image with matrix representation

On the other hand, a colored image consists of three channels, red, green, and blue, with each channel also containing pixel intensity values from 0 to 255. A colored image has a matrix shape of [height x width x channel]. In [Figure 35-3](#), we have an image of shape [10 x 10 x 3] indicating a 10 x 10 matrix with three channels.



**Figure 35-3.** Colored image with matrix representation

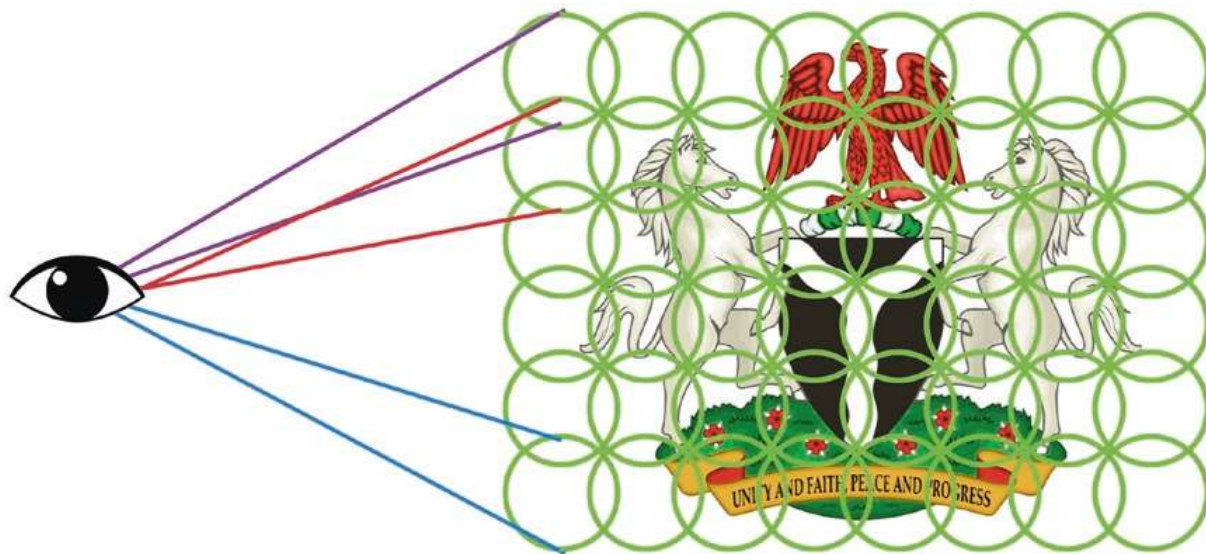
## Local Receptive Fields of the Visual Cortex

The core concept of convolutional neural networks is built on understanding the local receptive fields found in the neurons of the visual cortex – the part of the brain responsible for processing visual information.

A local receptive field is an area on the neuron that excites or activates that neuron to fire information to other neurons. When viewing an image, the neurons in the visual cortex react to a small or limited area of the overall image due to the presence of a small local receptive field.

Hence, the neurons in the visual cortex do not all sense the entire image at the same time, but they are activated by viewing a local area of the image via its local receptive field.

In Figure 35-4, the local receptive fields overlap to give a collective perspective on the entire image. Each neuron in the visual cortex reacts to a different type of visual information (e.g., lines with different orientations).



**Figure 35-4.** *Local receptive field*

Other neurons have large receptive fields that react to more complex visual patterns such as edges, regions, and so on. From here we get the idea that neurons with larger receptive field receive information from those with lower receptive fields as they progressively learn the visual information of the image.

## Advantages of CNN over MLP

Suppose we have a 28 x 28 pixel set of image data, a feedforward neural network or multilayer perceptron will need 784 input weights plus a bias. By flattening an image as you would in MLP, we lose the spatial relationship of the pixels in the image.

CNN, on the other hand, can learn complex image features by preserving the spatial relationship between the image pixels. It does so by stacking convolutional layers whereby the neurons in the higher layers with a larger receptive field receive information

from neurons in the lower layers having a smaller receptive field. CNN learns a hierarchy of increasingly complex features from the input data as it flows through the network.

In CNN, the neurons (or filters) in the convolutional layer are not all connected to the pixels in the input image as we have in the dense multilayer perceptron. Hence, a CNN is also called a sparse neural network.

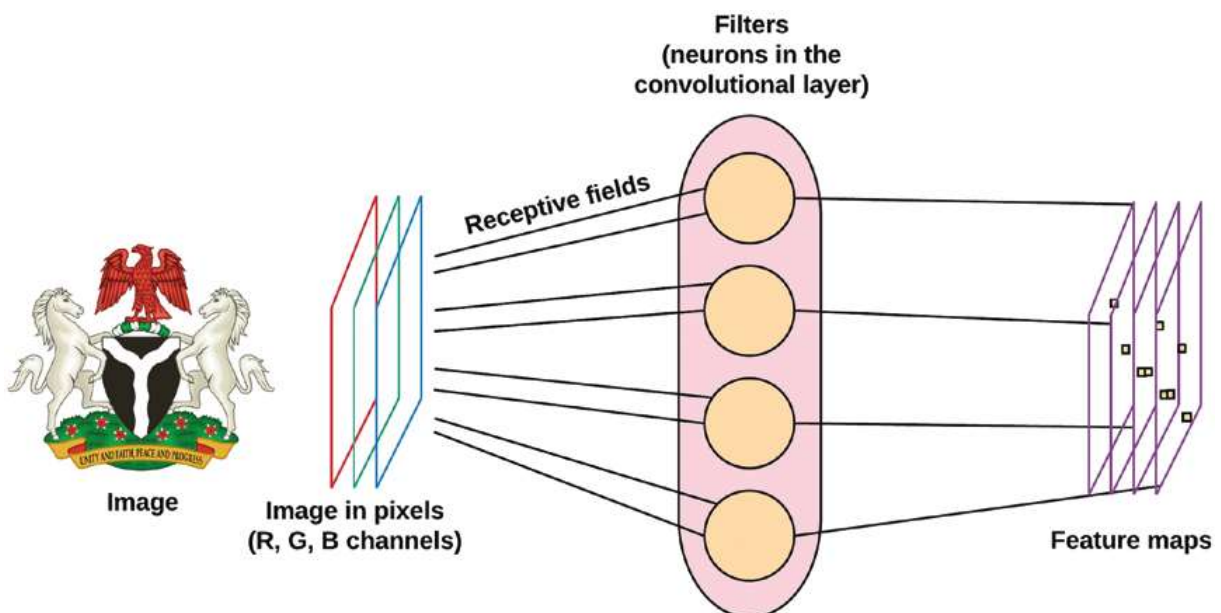
A distinct advantage of CNN over MLP is the reduced number of weights needed for training the network.

Convolutional neural networks are composed of three fundamental types of layers:

- Convolutional layer
- Pooling layer
- Fully connected layer

## The Convolutional Layer

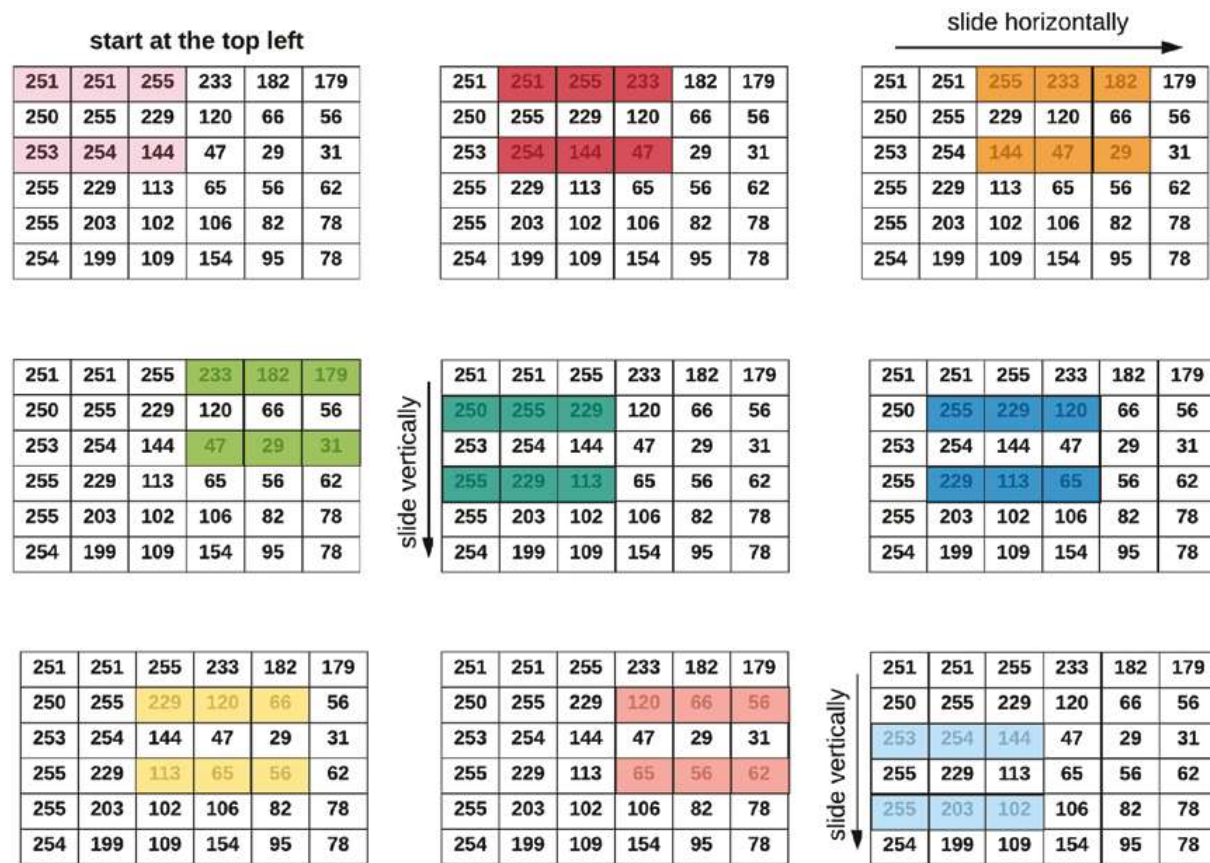
The convolution layer is made up of filters and feature maps. A filter is passed over the input image pixels to capture a specific set of features in a process called convolution (see Figure 35-5). The output of a filter is called a feature map.



**Figure 35-5.** *The convolution process*

## Convolution

Convolution is the process by which a function is applied to a matrix to extract specific information from the matrix. The function is implemented as a sliding window through the matrix, and it is more popularly called a convolutional filter or a kernel. Both terms are used interchangeably in the literature. The image in Figure 35-6 illustrates a filter sliding through a matrix to extract information from it.



**Figure 35-6.** Sliding window of a filter through a matrix

Filters are neurons in the convolutional layer. They are assigned weights and are applied as a sliding window through the matrix. The output of a filter is a feature map. Filters which are basically neurons also have a non-linear activation function.

The inputs into a filter can be the matrix of the image pixels if the filter is at the input layer, or it can be the feature maps of a previous convolutional layer if the filter is applied at a deeper layer in the network.

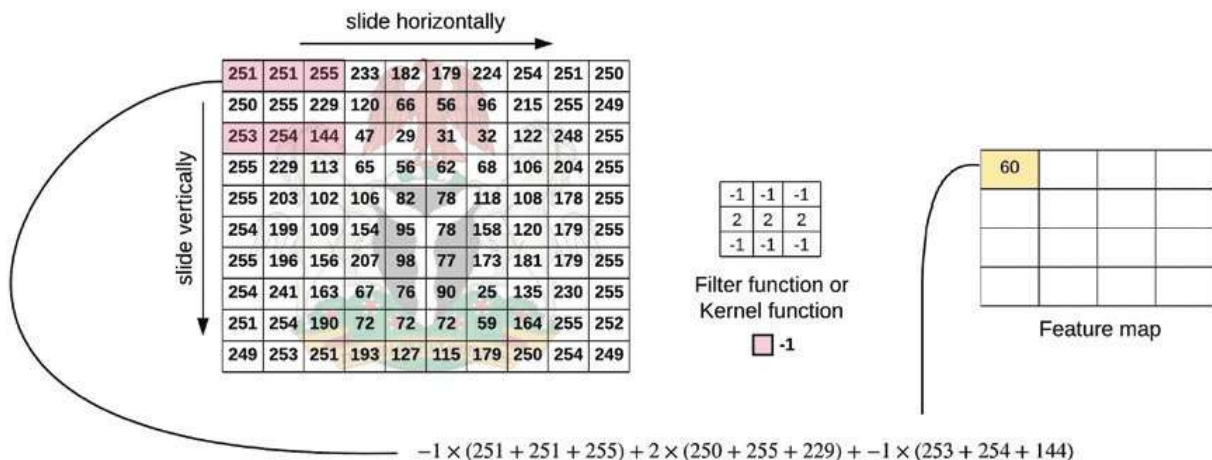
Filters are assigned a fixed square block for its input size. This input size can also be seen as the local receptive field of the filter. A common input size for filters is a 3 x 3 square patch as illustrated in Figure 35-7; other standard sizes include a 5 x 5 or 7 x 7 filter for extracting features from images. It is also a best practice to use more filters at deeper layers of the network and fewer filter at the input layer.

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| 2  | 2  | 2  |
| -1 | -1 | -1 |

A 3 x 3 Filter  
function or  
Kernel function

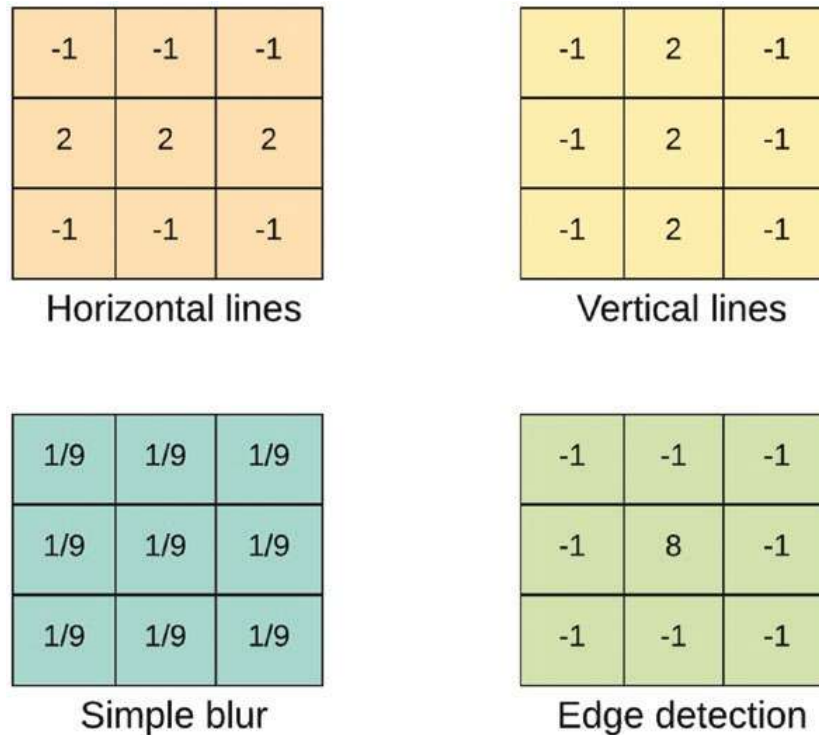
**Figure 35-7.** An example of a 3 x 3 filter kernel

Observe that each cell in the filter has an associated weight or value. These values are used to multiply its associated pixel intensities and then sum up their results, which are filled in the appropriate cell of the convolutional result. This procedure is illustrated in Figure 35-8.



**Figure 35-8.** Sliding a convolutional filter across an image matrix to extract features

The weights on the filter determine the filter operation and consequently the type of features that are extracted from the filter inputs. Different filters are responsible for edge detection, line detection, and so on. See Figure 35-9.



**Figure 35-9.** Filter types

Key considerations to make when designing a convolutional layer are

- The filter size
- The stride of the filter
- The padding for the layer input

The *stride* of the filter determines how many pixel steps the filter makes when moving from one image activation to another. It is typical to use a stride of 1, although this could be increased for large images. See Figure 35-10.

**slide with stride = 1**

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 251 | 251 | 255 | 233 | 182 | 179 |
| 250 | 255 | 229 | 120 | 66  | 56  |
| 253 | 254 | 144 | 47  | 29  | 31  |
| 255 | 229 | 113 | 65  | 56  | 62  |
| 255 | 203 | 102 | 106 | 82  | 78  |
| 254 | 199 | 109 | 154 | 95  | 78  |

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 251 | 251 | 255 | 233 | 182 | 179 |
| 250 | 255 | 229 | 120 | 66  | 56  |
| 253 | 254 | 144 | 47  | 29  | 31  |
| 255 | 229 | 113 | 65  | 56  | 62  |
| 255 | 203 | 102 | 106 | 82  | 78  |
| 254 | 199 | 109 | 154 | 95  | 78  |

**slide with stride = 2**

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 251 | 251 | 255 | 233 | 182 | 179 |
| 250 | 255 | 229 | 120 | 66  | 56  |
| 253 | 254 | 144 | 47  | 29  | 31  |
| 255 | 229 | 113 | 65  | 56  | 62  |
| 255 | 203 | 102 | 106 | 82  | 78  |
| 254 | 199 | 109 | 154 | 95  | 78  |

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 251 | 251 | 255 | 233 | 182 | 179 |
| 250 | 255 | 229 | 120 | 66  | 56  |
| 253 | 254 | 144 | 47  | 29  | 31  |
| 255 | 229 | 113 | 65  | 56  | 62  |
| 255 | 203 | 102 | 106 | 82  | 78  |
| 254 | 199 | 109 | 154 | 95  | 78  |

**Figure 35-10.** *An illustration of stride width*

Sometimes the choice of our filter size and the selected stride may not evenly divide up the size of the input to the filter. So to avoid losing pixel information since we don't slide past the edge of the image, a technique called *zero padding* is employed to pad the borders of the image pixels with a defined layer of zeros. This allows the filter to stride evenly through all the pixels in the image by including the zeros in the convolution. See Figure 35-11.

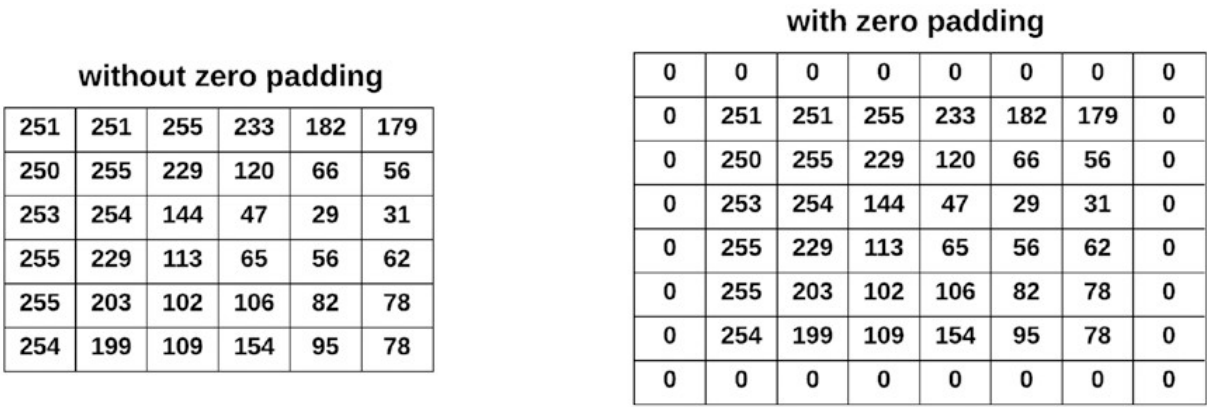


Figure 35-11. An illustration of zero padding

Feature Maps

Feature maps are the outputs of a filter in a convolutional layer. Feature maps bring to the fore certain patterns of the input image such as horizontal lines, vertical lines, edges, and so on. These feature maps of the various neurons stacked together are what forms a convolutional neural layer and enable the layer to learn complex patterns and features of an image.

Moving deeper across the convolutional neural network, the inputs to a deeper convolutional layer are the feature maps of the previous layer. See Figure 35-12.

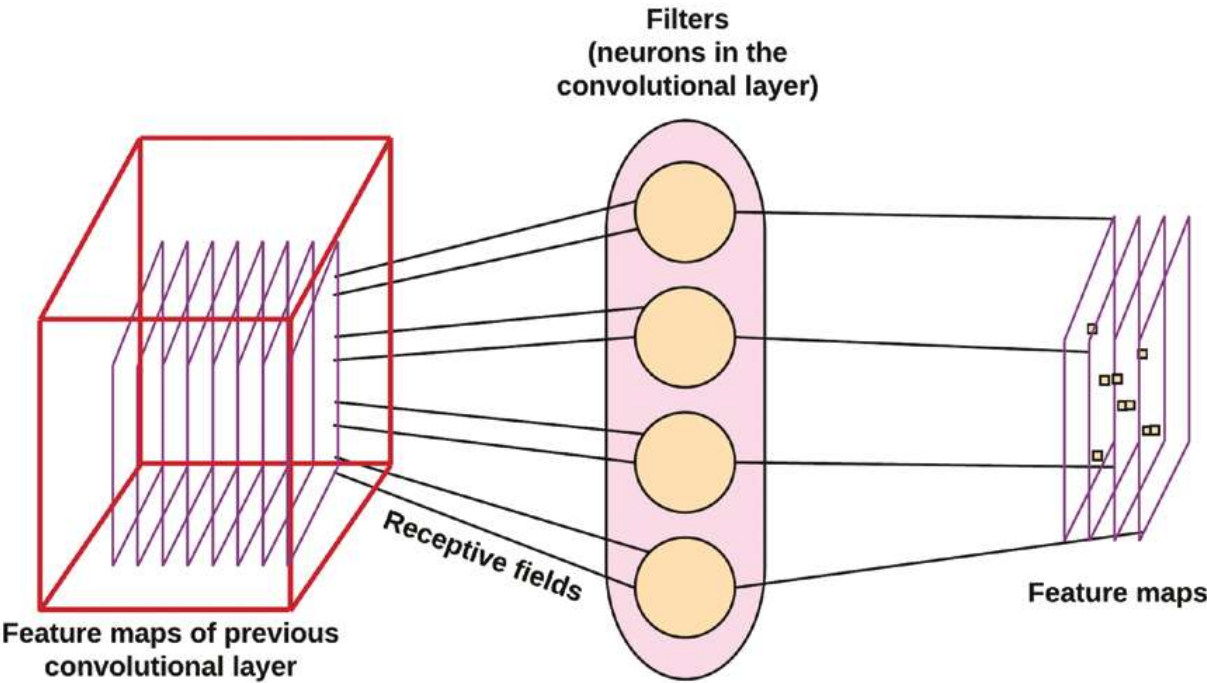


Figure 35-12. Feature maps as inputs to a convolutional layer

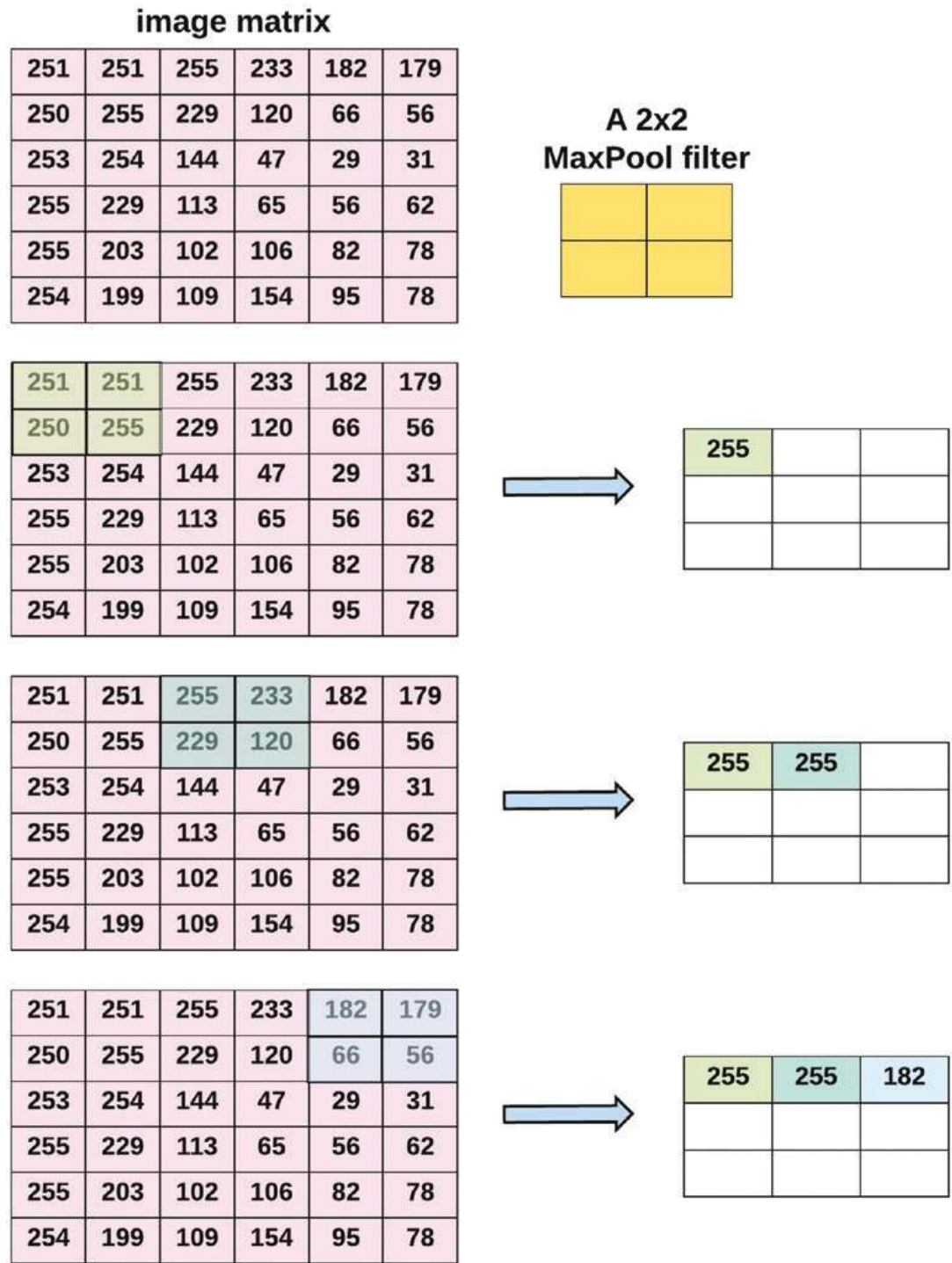
## The Pooling Layer

Pooling layers typically follow one or more convolutional layers. The goal of the pooling layer is to reduce or downsample the feature map of the convolutional layer. The pooling layer summarizes the image features learned in the previous network layers. By doing so, it also helps prevent the network from overfitting. Moreover, the reduction in the input size also bodes well for processing and memory costs when training the network.

The pooling layer can be seen as an aggregation function that consolidates learned features and extracts the essential features from previous layers. It does not conduct any multiplicative transformation on the input feature maps as seen in the convolutional layer.

The aggregation functions carried out by the pooling layer include max, sum, and average. The most frequently used aggregation function in practice is the max and is commonly called the MaxPool.

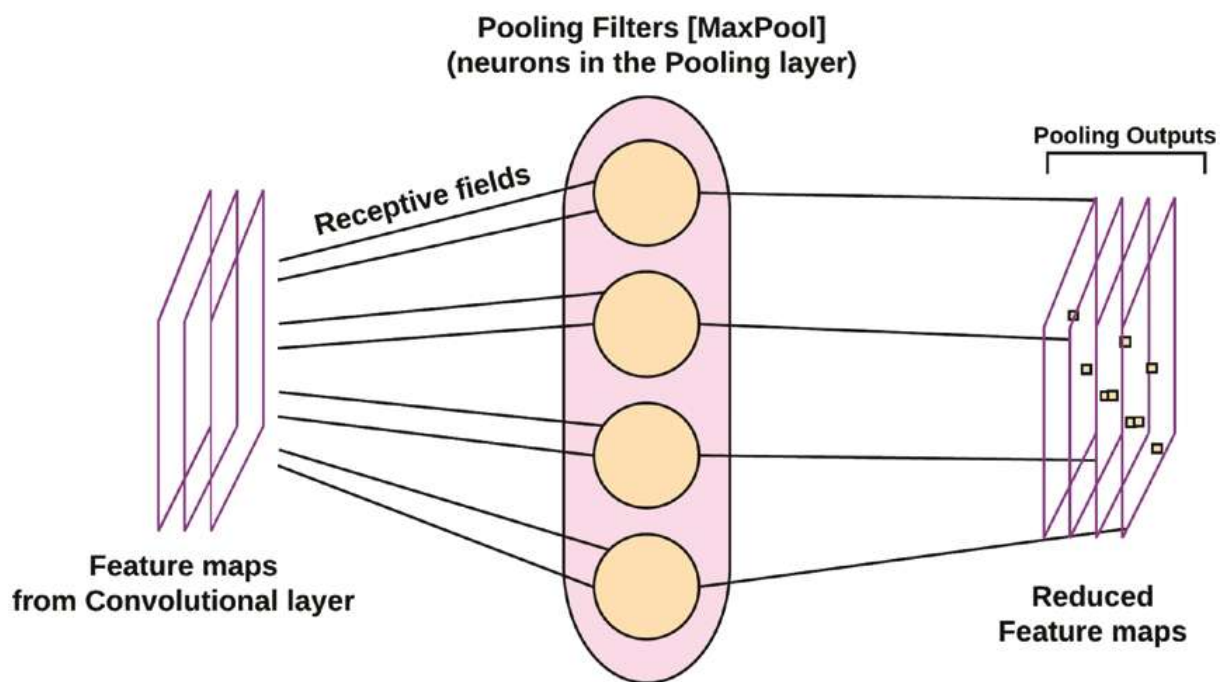
The aggregation functions of the pooling layer serve as the layers' filters. Just like the filters of the convolutional layer, they have a receptive field (although smaller in size than that of the convolutional layer) and a stride width. However, the filters which are the neurons of the pooling layer have no weight or biases. A typical size for the pooling filter is a 2 x 2 matrix as shown in Figure 35-13.



*Figure 35-13. Example of pooling with MaxPooling*

The essential advantage of the pooling layer is its ability to inject location invariance into the network. Location invariance means that features can be detected by the network no matter where they are on the image.

The pooling layer applies its aggregation function to all the channels of the input image. For example, in an R, G, B image (i.e., an image with three channels, red, green, and blue), the MaxPool will be applied independently to all the three channels. Similarly, for feature maps with a particular depth, the pooling aggregation will be applied separately to each feature map. See Figure 35-14 as an example of applying pooling to the channel depth of its inputs.



**Figure 35-14.** Example of applying pooling to input with depth. Note that the filters in the pooling layer have no weights or biases

## The Fully Connected Network Layer

The fully connected network (FCN) layer is our regular feedforward neural network or multilayer perceptron. These layers typically have a non-linear activation function. In any case, the FCN is the final layer of the convolutional neural network. In this case, a softmax activation is used to output the probabilities that an input belongs to a particular class.

Before passing an input into the FCN, the image matrix will have to be flattened. For example, a  $28 \times 28 \times 3$  image matrix will become 2352 input weights plus a bias of 1 into the fully connected network.

In the case of our convolutional network, the feature maps of either the convolutional or pooling layer are flattened before passing into the FCN to compute the final network probabilities using the softmax function.

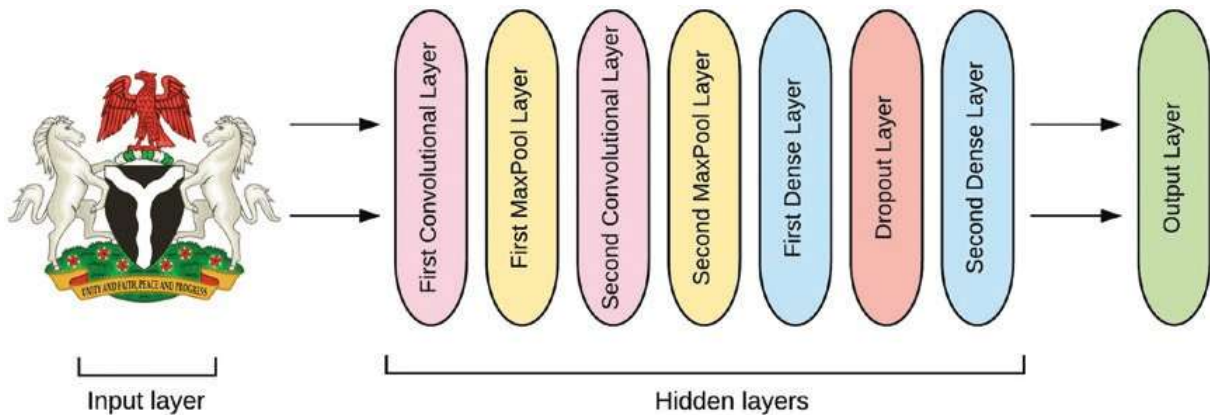
## An Example CNN Architecture

We have discussed the building blocks of a convolutional neural network system. As you've seen, a CNN system is principally composed of convolution layers, pooling layers, and the fully connected layer. However, the way these layers are arranged and in what number are down to the preferred heuristics of the particular use case that a CNN is employed in solving.

An example CNN modeling pipeline is shown here:

1. The first layer following the input layer of images must be a convolutional layer for extracting image features. A  $3 \times 3$  image filter is commonly used depending on the size of the input image.
2. Pooling layers typical follow a set of one or more convolutional layers. Typically, a  $2 \times 2$  filter size is used in the pooling layer.
3. The fully connected layer must be the final layer of the CNN. It is also called the dense layer. It contains the softmax activation function to give the probabilities of class membership.
4. CNN may include one or more Dropout layers to prevent the network from overfitting.

Figure 35-15 is an example of a CNN architecture.



**Figure 35-15.** CNN architecture

## CNN for Image Recognition with TensorFlow 2.0

In this example, we will build a convolutional neural network (CNN) to classify images from the CIFAR-10 dataset. CIFAR-10 is another standard image classification dataset to classify a colored 32 x 32 pixel image data into ten image classes, namely, airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The focus of this section is exclusively on using TensorFlow 2.0 methods to build a CNN image classifier.

The CNN model architecture implemented loosely mirrors the Krizhevsky's architecture, also known as AlexNet. The network architecture has the following layers:

- Convolution layer: `kernel_size = [5 x 5]`
- Convolution layer: `kernel_size = [5 x 5]`
- Batch normalization layer
- Convolution layer: `kernel_size = [5 x 5]`
- Max pooling: `pool size = [2 x 2]`
- Convolution layer: `kernel_size = [5 x 5]`
- Convolution layer: `kernel_size = [5 x 5]`
- Batch normalization layer
- Max pooling: `pool size = [2 x 2]`

- Convolution layer: kernel\_size = [5 x 5]
- Convolution layer: kernel\_size = [5 x 5]
- Convolution layer: kernel\_size = [5 x 5]
- Max pooling: pool size = [2 x 2]
- Dropout layer
- Dense layer: units = [512]
- Dense layer: units = [256]
- Dropout layer
- Dense layer: units = [10]

This CNN model has close to a million trainable variables as can be seen from the model summary when running **model.summary()**. Training on a CPU will take an inordinate amount of time (about 1 hour and 30 minutes). For this code example, we will train on a GPU instance. If running the code on Google Colab, change the runtime type to GPU and install TensorFlow 2.0 with GPU package. The graph of the model in Tensorboard is shown in Figure 35-16.

```
# import TensorFlow 2.0 with GPU
!pip install -q tf-nightly-gpu-2.0-preview

# import packages
import tensorflow as tf

# confirm tensorflow can see GPU
device_name = tf.test.gpu_device_name()
if device_name != '/device:GPU:0':
    raise SystemError('GPU device not found')
print('Found GPU at: {}'.format(device_name))

# load the TensorBoard notebook extension
%load_ext tensorboard

# import dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# change datatype to float
```

```

x_train = x_train.astype('float32')
x_test = x_test.astype('float32')

# scale the dataset from 0 -> 255 to 0 -> 1
x_train /= 255
x_test /= 255

# one-hot encode targets
y_train = tf.keras.utils.to_categorical(y_train)
y_test = tf.keras.utils.to_categorical(y_test)

# parameters
batch_size = 100

# create dataset pipeline
train_ds = tf.data.Dataset.from_tensor_slices(
    (x_train, y_train)).shuffle(len(x_train)).repeat().batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((x_test, y_test)).batch(batch_size)

# create the model
def model_fn():
    model_input = tf.keras.layers.Input(shape=(32, 32, 3))
    x = tf.keras.layers.Conv2D(64, (5, 5), padding='same',
        activation='relu')(model_input)
    x = tf.keras.layers.Conv2D(64, (5, 5), padding='same', activation='relu')(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Conv2D(64, (5, 5), padding='same', activation='relu')(x)
    x = tf.keras.layers.MaxPooling2D(pool_size=(2, 2), strides=2,
        padding='same')(x)
    x = tf.keras.layers.Conv2D(64, (5, 5), padding='same', activation='relu')(x)
    x = tf.keras.layers.Conv2D(64, (5, 5), padding='same', activation='relu')(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.MaxPooling2D(pool_size=(2, 2), strides=2, padding='same')(x)
    x = tf.keras.layers.Conv2D(32, (3, 3), padding='same', activation='relu')(x)
    x = tf.keras.layers.Conv2D(32, (3, 3), padding='same', activation='relu')(x)
    x = tf.keras.layers.Conv2D(32, (3, 3), padding='same', activation='relu')(x)
    x = tf.keras.layers.MaxPooling2D(pool_size=(2, 2), strides=2,
        padding='same')(x)

```

```

x = tf.keras.layers.Dropout(0.3)(x)
x = tf.keras.layers.Flatten()(x)
x = tf.keras.layers.Dense(512, activation='relu')(x)
x = tf.keras.layers.Dense(256, activation='relu')(x)
x = tf.keras.layers.Dropout(0.5)(x)
output = tf.keras.layers.Dense(10, activation='softmax')(x)

# the model
model = tf.keras.Model(inputs=model_input, outputs=output)

# compile the model
model.compile(optimizer=tf.keras.optimizers.Nadam(),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

return model

# build the model
model = model_fn()

# print model summary
model.summary()

# tensorboard
tensorboard = tf.keras.callbacks.TensorBoard(log_dir='./tmp/logs_cifar10_
keras',
                                             histogram_freq=0, write_graph=True,
                                             write_images=True)

# assign callback
callbacks = [tensorboard]

# train the model
history = model.fit(train_ds, epochs=10,
                   steps_per_epoch=500,
                   callbacks=callbacks)

# evaluate the model
score = model.evaluate(test_ds)
print('Test loss: {:.2f} \nTest accuracy: {:.2f}%'.format(score[0], score[1]*100))

```

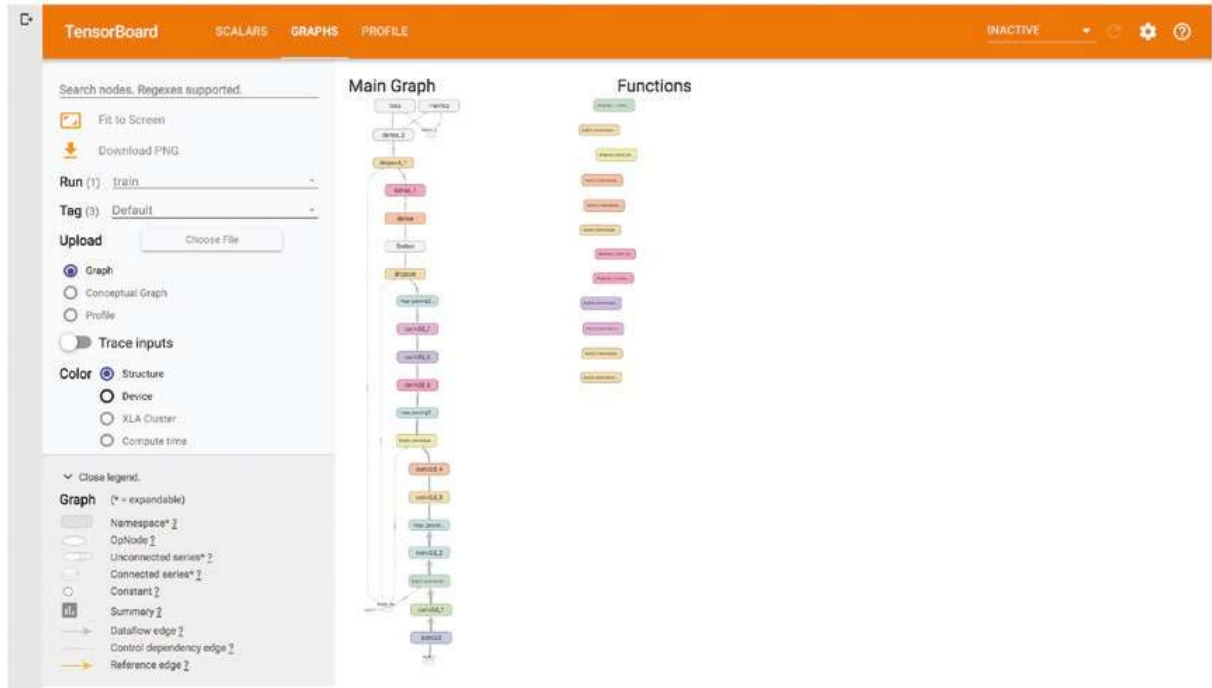
```
'Output:'
```

```
Test loss: 0.74
```

```
Test accuracy: 80.05%
```

```
# execute the command to run TensorBoard
```

```
%tensorboard --logdir tmp/logs_cifar10_keras
```



**Figure 35-16.** Tensorboard output of CIFAR-10 model graph

In this chapter, we discussed the convolutional neural network (CNN) as an example of a deep neural network. We went through the design details in architecting a CNN and implemented a CNN model with TensorFlow 2.0. In the next chapter, we will examine another type of deep neural network called the recurrent neural network.